

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/19

REQ005 (Precio Variable):

Función que permite digitar manualmente conceptos y montos libres para consumos de papelería, validando rigurosamente que el valor ingresado sea un número decimal positivo.

1. CÓDIGO FUENTE para Precio Variable Controlador (Código)

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.view.VariableServicePanel;

import javax.swing.*;
import java.util.ArrayList;
import java.util.List;

public class VariableServiceController {

    private VariableServicePanel view;
    private CashFlowModel cashFlowModel;

    private List<VariableServiceItem> temporalServices;

    private static class VariableServiceItem {
        private String concept;
        private double amount;

        public VariableServiceItem(String concept, double amount) {
            this.concept = concept;
            this.amount = amount;
        }
    }

    public VariableServiceController(VariableServicePanel view, CashFlowModel cashFlowModel) {
        this.view = view;
        this.cashFlowModel = cashFlowModel;
        this.temporalServices = new ArrayList<>();

        initEvents();
        actualizarResumenTemporal();
    }

    private void initEvents() {
        view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());

        view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
    }

    private void agregarServicioTemporal() {
        String concepto = view.getConcepto().trim();
        String valorTexto = view.getValor().trim().replace(",", ".");
```

```

if (concepto.isEmpty() || valorTexto.isEmpty()) {
    showError("Debe ingresar el concepto y el valor del servicio.");
    return;
}

double valor;

try {
    valor = Double.parseDouble(valorTexto);
} catch (NumberFormatException ex) {
    showError("El valor debe ser numérico. Ejemplo: 3.50");
    return;
}

if (valor <= 0) {
    showError("El valor debe ser mayor a cero.");
    return;
}

temporalServices.add(new VariableServiceItem(concepto, valor));

view.limpiarCampos();
actualizarResumenTemporal();
}

private void guardarServiciosEnCaja() {
    if (temporalServices.isEmpty()) {
        showError("No hay servicios variables para guardar.");
        return;
    }

    try {
        for (VariableServiceItem item : temporalServices) {
            cashFlowModel.addVariableServiceSale(item.concepto, item.amount);
        }

        temporalServices.clear();
        actualizarResumenTemporal();

        JOptionPane.showMessageDialog(
            view,
            "Servicios variables guardados correctamente en el cierre de caja.",
            "Servicios guardados",
            JOptionPane.INFORMATION_MESSAGE
        );
    } catch (IllegalArgumentException ex) {
        showError(ex.getMessage());
    }
}

```

```

private void actualizarResumenTemporal() {
    if (temporalServices.isEmpty()) {
        view.setResumenTemporal("No hay servicios variables agregados.");
        return;
    }

    StringBuilder sb = new StringBuilder();
    double subtotal = 0.0;

    sb.append("SERVICIOS VARIABLES TEMPORALES\n\n");

    for (VariableServiceItem item : temporalServices) {
        subtotal += item.amount;

        sb.append(String.format(
            "%s = $%.2f\n",
            item.concept,
            item.amount
        ));
    }

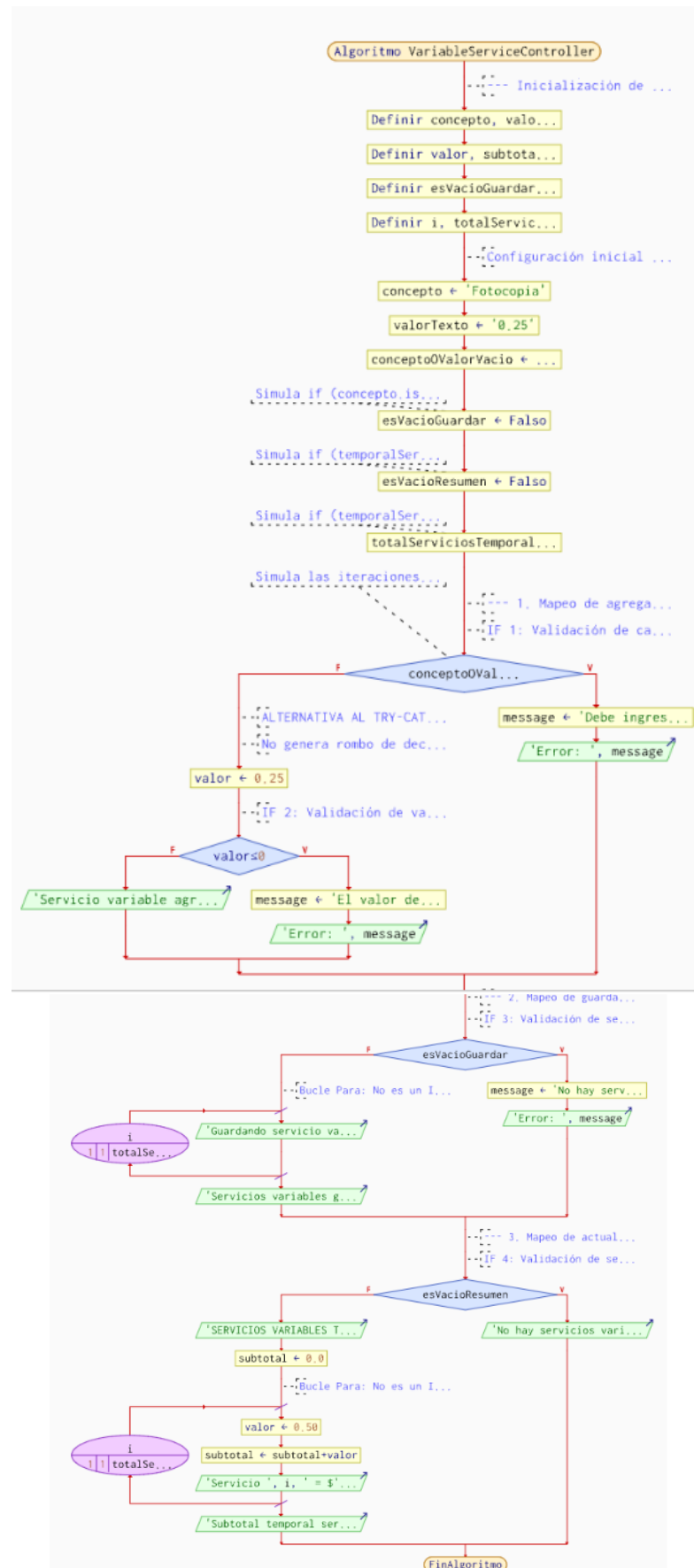
    sb.append(String.format("\nSubtotal temporal servicios variables: $%.2f", subtotal));

    view.setResumenTemporal(sb.toString());
}

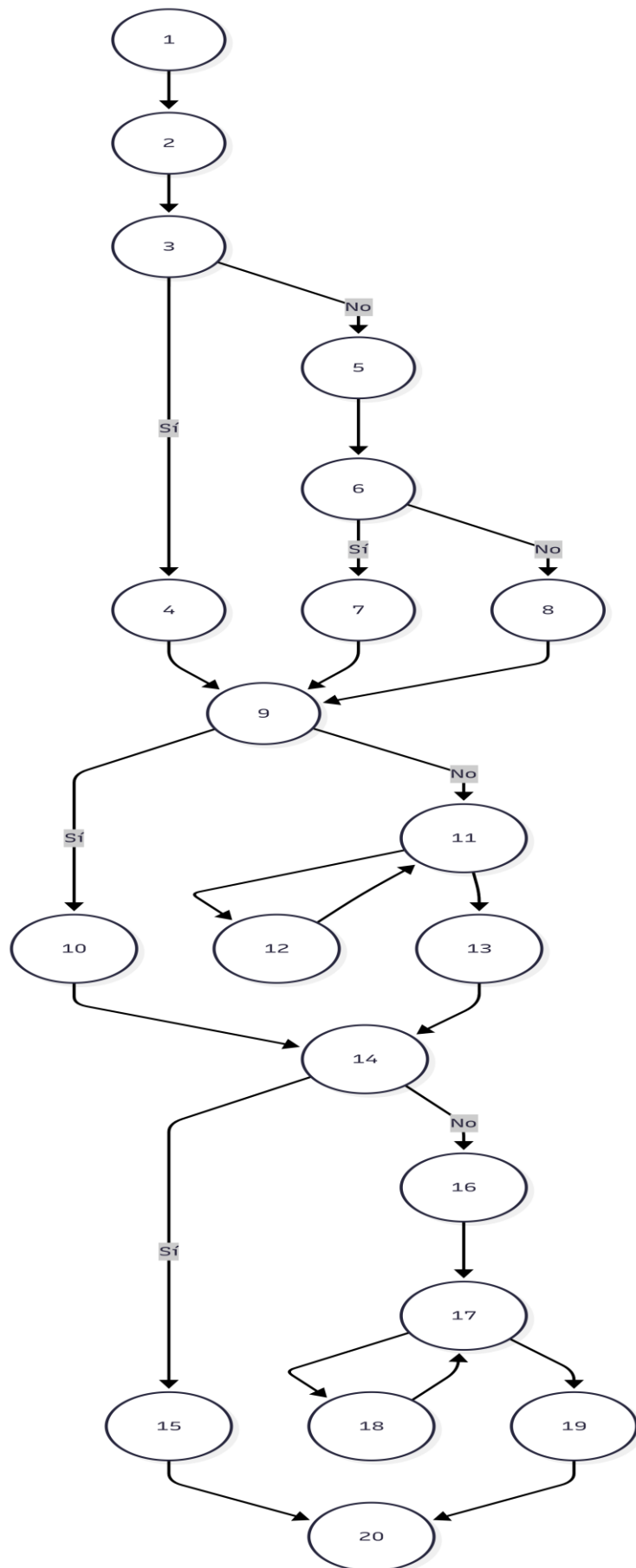
private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}

```

2. DIAGRAMA DE FLUJO (DF) Precio Variable



3. GRAFO DE FLUJO (GF) Precio Variable



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Precio Variable

Determinar en base al GF del numeral 4
RUTAS

R1: 1-2-3-4-9-10-14-15-20

R2: 1-2-3-5-6-7-9-10-14-15-20

R3: 1-2-3-5-6-8-9-10-14-15-20

R4: 1-2-3-4-9-11-12-11-13-14-15-20

R5: 1-2-3-4-9-10-14-16-17-18-17-19-20

5. COMPLEJIDAD CICLOMÁTICA Precio Variable

Se puede calcular de las siguientes formas:

V(G) Nodos predcados (IF-THEN-ELSE) +1

$V(G) = 4 + 1$

$V(G) = 5$

$V(G) = A - N + 2$

$V(G) = 23 - 20 + 2$

$V(G) = 5$

DONDE:

P: Número de nodos predcado

A: Número de aristas

N: Número de nodos